

**[SOFTWARE EDIFICIO MIRADOR]
(DAS) Documento Arquitectura de Software
Versión 1.0**

Identificación de Documento

Identificación	EM1
Proyecto	SOFTWARE EDIFICIO MIRADOR
Versión	1.3
Documento mantenido por	Consuelo Betancur, Eduardo tapia, Álvaro Salazar
Fecha de última revisión	31/03/2026
Fecha de próxima revisión	02/04/2026
Documento aprobado por	Consuelo Betancur, Eduardo tapia, Álvaro Salazar
Fecha de última aprobación	

Historia de Revisiones

Fecha	Versión	Descripción	Autor
30/03/2026	1.0	Empezamos el documento DAS y se hablo como grupo que se hace en el ítem 1, logramos hacer 3 contenidos	Consuelo Betancur, Eduardo tapia, Álvaro Salazar
31/03/2026	1.1	Continuamos el documento DAS, y logramos resumir o modificar cosas anteriores para que se entienda mejor	Consuelo Betancur, Eduardo tapia, Álvaro Salazar
31/03/2026	1.2	Terminamos el primer ITEM y continuamos con el segundo ITEM	Consuelo Betancur, Eduardo tapia, Álvaro Salazar
	1.3		

Tabla de Contenidos

1. INTRODUCCIÓN.....	4
1.1. CONTEXTO DEL PROBLEMA	4
1.2. PROPÓSITO	4
1.3. ÁMBITO	4
1.4. DEFINICIONES, ACRÓNIMOS Y ABREVIACIONES	4
1.5. RESUMEN EJECUTIVO.....	4
1.6. ARQUITECTURA DEL SISTEMA	4
2. VISIÓN DEL SISTEMA.....	4
2.1. DESCRIPCIÓN GENERAL DEL SISTEMA	4
2.2. OBJETIVOS DEL SISTEMA	4
2.3. REQUERIMIENTOS FUNCIONALES	4
2.4. REQUERIMIENTOS NO FUNCIONALES	5
2.5. SUPUESTOS Y DEPENDENCIAS	5
3. ESTILOS Y PATRONES ARQUITECTÓNICOS	5
3.2. JUSTIFICACIÓN DEL ESTILO SEGÚN EL CONTEXTO DEL SISTEMA	5
4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS	5
4.1. VISTA DE ESCENARIO	5
4.1.1. <i>Propósito</i>	5
4.1.2. <i>Actores</i>	5
4.1.3. <i>Diagrama general de casos de uso</i>	5
4.1.4. <i>Diagrama de casos de uso específicos</i>	5
4.1.6. <i>Especificación de casos de uso</i>	5
4.2. VISTA LÓGICA.....	6
4.2.1. <i>Propósito</i>	6
4.2.2. <i>Diagrama de clases</i>	6
4.2.3. <i>Descripción diagrama de clases</i>	6
4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO	7
4.3.1. <i>Propósito</i>	7
4.3.2. <i>Diagrama de componente</i>	7
4.3.3. <i>Descripción diagrama de componente</i>	7
4.3.4. <i>Diagrama de paquete</i>	7
4.3.5. <i>Descripción diagrama de paquete</i>	7
4.4. VISTA DE PROCESOS.....	7
4.4.1. <i>PROPÓSITO</i>	7
4.4.2. <i>DIAGRAMA DE ACTIVIDAD</i>	7
4.4.3. <i>DESCRIPCIÓN DIAGRAMA DE ACTIVIDAD</i>	7
4.5. VISTA FÍSICA.....	7
4.5.1. <i>Propósito</i>	7
4.5.2. <i>Diagrama de despliegue</i>	7
4.5.3. <i>Descripción diagrama de despliegue</i>	7
5. REQUISITOS DE CALIDAD.....	7

5.1. PROPÓSITO	7
5.3. <i>Reglas y criterios de evaluación de calidad</i>	7
6. PRINCIPIOS DE DISEÑO APLICADOS	8
6.1. <i>Propósito</i>	8
6.2. PRINCIPIOS DE DISEÑO (POR EJEMPLO: ABSTRACCIÓN, ACOPLAMIENTO, COHESIÓN, ENCAPSULAMIENTO, MODULARIDAD).....	8
7. PROTOTIPO	8
7.1. PROPÓSITO	8
7.2. MOCKUPS (IMÁGENES CON UNA BREVE DESCRIPCIÓN)	8
7.3. JUSTIFICAR HERRAMIENTAS DE PROTOTIPADO	8
8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN	8
8.1. PROPÓSITO	8
8.2. LISTA DE VERIFICACIÓN	8
8.3. ANÁLISIS Y MÉTRICAS DE RESULTADOS	8
9. CONTROL DE VERSIONES	8
9.1. PROPÓSITO	8
9.2. CONTROL DE VERSIÓN UTILIZADO (JUSTIFICAR EL TIPO DE CONTROL DE VERSIÓN UTILIZAD (FECHA, SEMÁNTICA O SECUENCIAL).....	8
9.3. JUSTIFICAR HERRAMIENTAS DE VERSIONAMIENTO	8
7. CONCLUSIONES	8
8. BIBLIOGRAFÍA	8
9. ANEXOS	8
9.1. CARTA GANTT	8

1. INTRODUCCIÓN

1.1. Contexto del Problema

El problema se sitúa en que una comunidad de gran tamaño, ‘Edificio Mirador’, depende actualmente de un sistema de gestión obsoleto. Esta situación genera una serie de dificultades, entre las que destacan la falta de confianza por parte de los residentes, la ineficiencia en los procesos de cobro y un aumento en los niveles de morosidad, debido principalmente a la inexistencia de un mecanismo eficaz para identificar y monitorear oportunamente a quienes cumplen con sus pagos y a quienes presentan atrasos.

La raíz del problema radica en el **procesamiento manual de datos y la falta de una base de datos centralizada**, lo que impide la integridad de la información y genera cuellos de botella en la validación de cobros. Técnicamente, la inexistencia de algoritmos de cálculo automatizado para intereses y la carencia de módulos de notificación en tiempo real provocan errores humanos recurrentes, imposibilitando una gestión financiera transparente y escalable para una comunidad de gran tamaño.

1.2. Propósito

Implementar un sistema de gestión moderno y automatizado que permita mejorar el control administrativo y financiero del edificio, facilitando la toma de decisiones, el seguimiento de pagos y la gestión eficiente de la información.

1.3. Ámbito

El proyecto se centra en modernizar la administración del edificio "El Mirador", optimizando la gestión de pagos, disminuyendo la morosidad, agilizando procesos internos y fortaleciendo la comunicación con los residentes a través del seguimiento de solicitudes y reclamos, asegurando un funcionamiento más eficiente y transparente.

1.4. Definiciones, acrónimos y abreviaciones

ACRONIMO	DESCRIPCION

1.5. Resumen ejecutivo

El Edificio Mirador padece una crisis de morosidad y desconfianza causada por un sistema de gestión manual y obsoleto que impide la integridad de los datos. Se propone implementar una solución tecnológica automatizada bajo una arquitectura MVC, diseñada para centralizar la información y eliminar errores humanos en el cálculo de deudas.

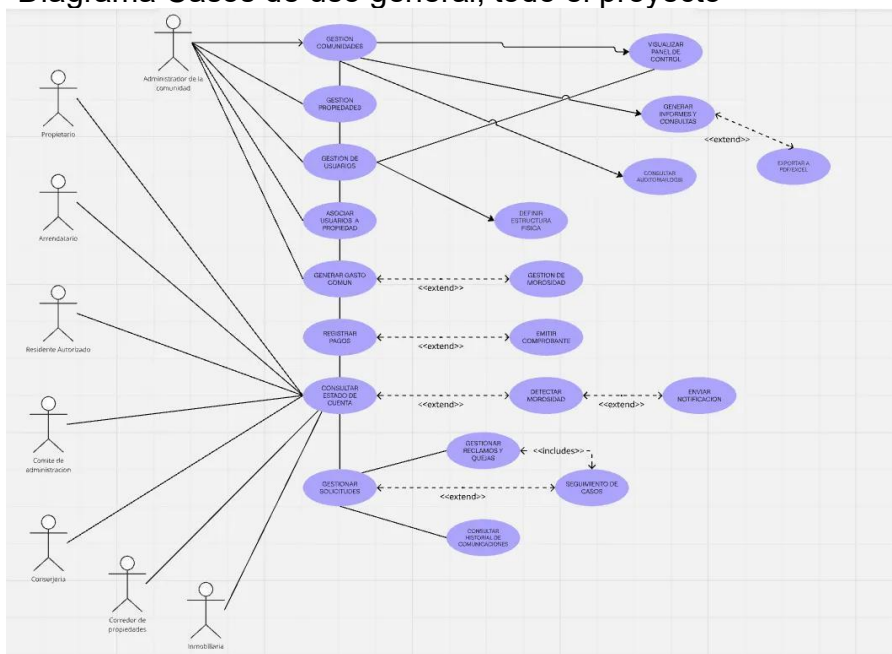
El sistema define dos perfiles críticos: el Administrador, responsable de la gestión financiera y reportes de cobranza, y el Residente, quien podrá consultar estados de cuenta y gestionar reclamos en tiempo real. Con esta modernización, se garantiza la trazabilidad total de los fondos, una comunicación interna fluida y una reducción drástica de la morosidad mediante notificaciones automáticas.

1.6. Arquitectura del sistema

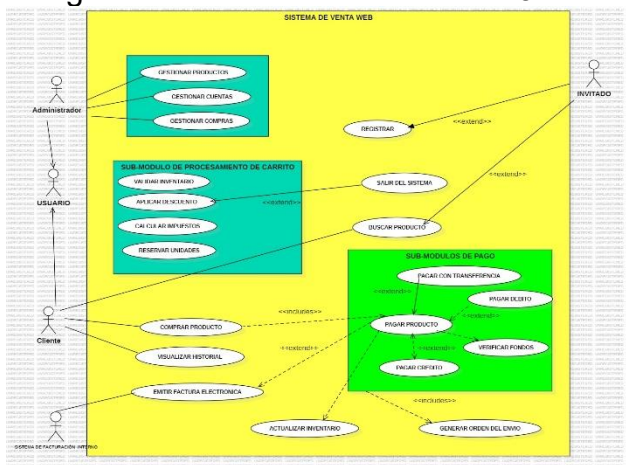
(ej. vista de escenario, vista lógica, vista de desarrollo, vista de proceso, vista física)

VISTA ESCENARIO

-Diagrama Casos de uso general, todo el proyecto

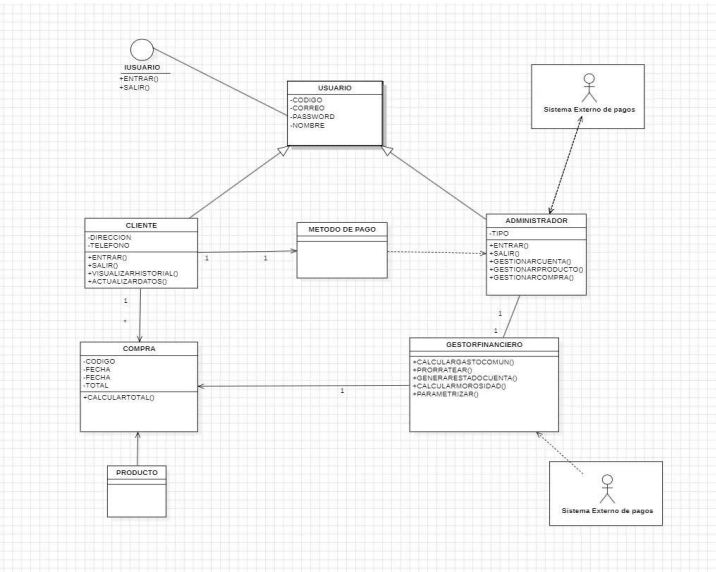


-Diagrama caso de uso subsistema 3



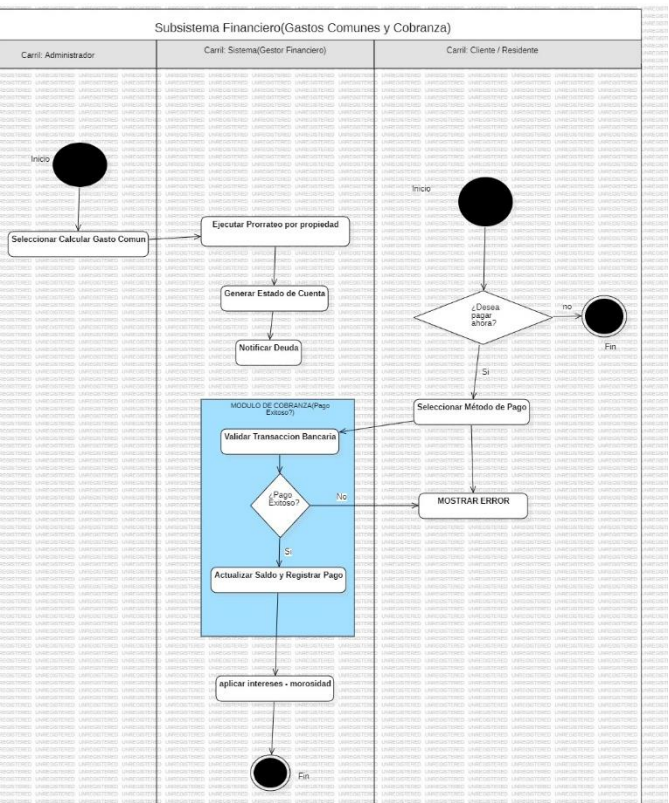
VISTA LOGICA

-Diagrama de clases



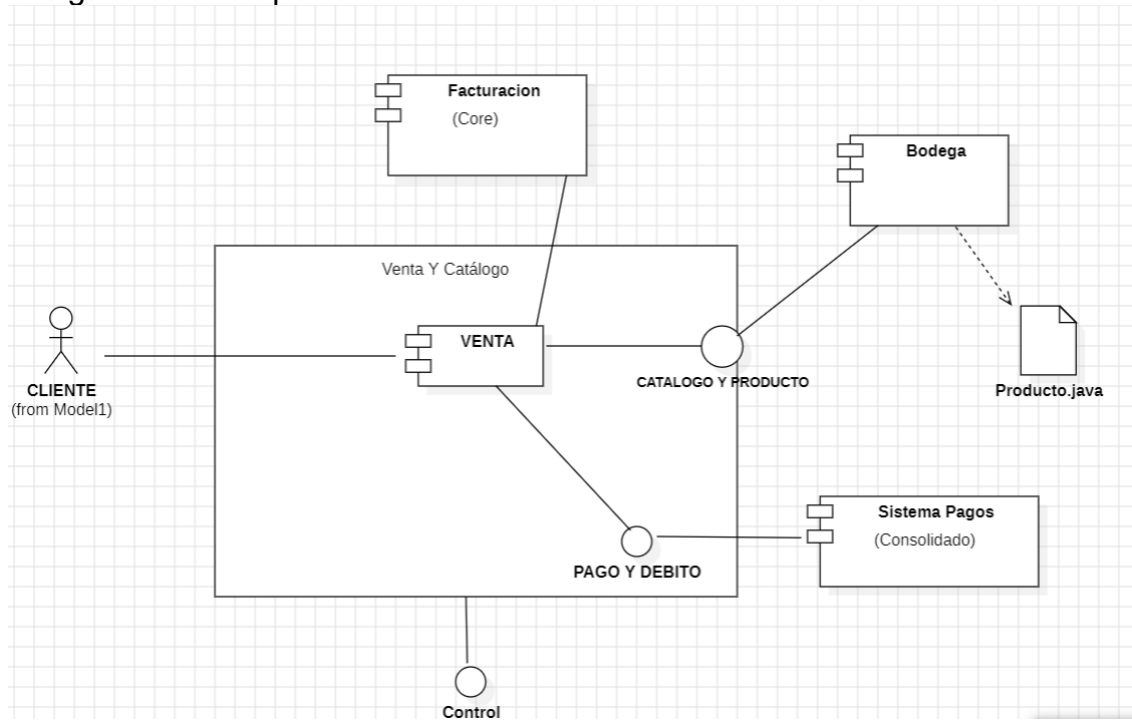
VISTA DE PROCESOS

-Diagrama de actividades

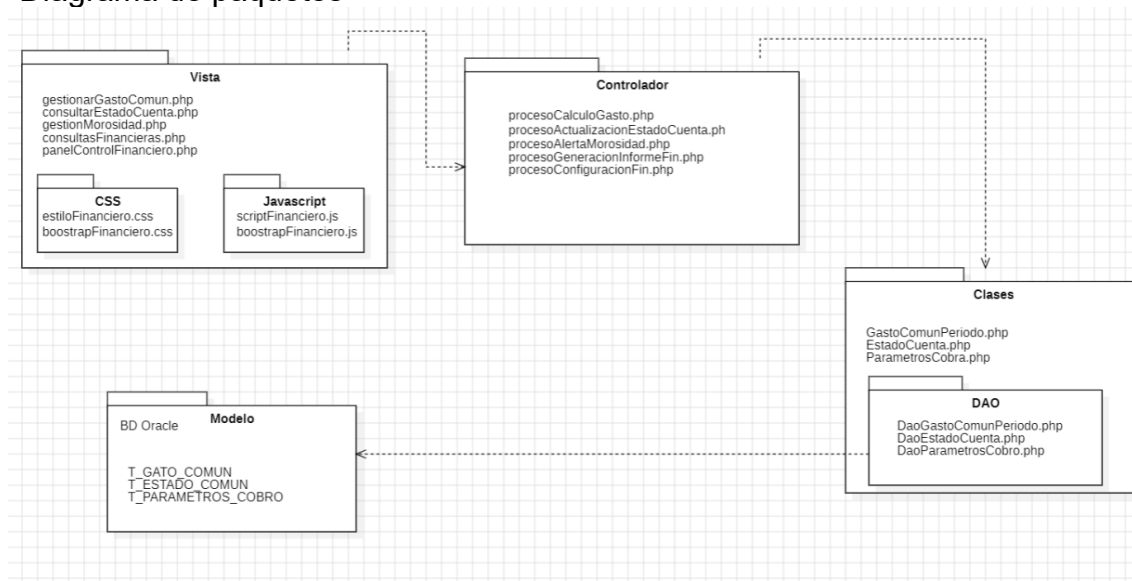


VISTA DESARROLLO

-Diagrama de componentes

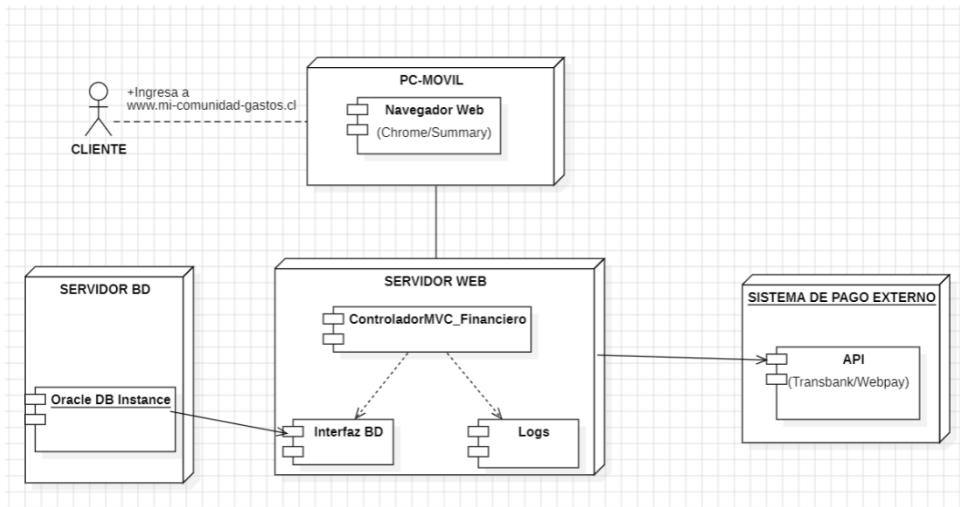


-Diagrama de paquetes



VISTA FÍSICA

-Diagrama de despliegue



2. VISIÓN DEL SISTEMA

2.1. Descripción general del sistema

El sistema es una plataforma de gestión que permitirá administrar residentes, registrar pagos de gastos comunes, controlar la morosidad y generar reportes financieros, la organización y transparencia en la administración del edificio Mirador.

2.2. Objetivos del sistema

Desarrollar e implementar un sistema tecnológico que permita gestionar de manera eficiente y automatizada los pagos de las cuotas de gastos comunes, el registro de residentes, el seguimiento de solicitudes y quejas, y la generación de informes, garantizando transparencia, reducción de morosidad y mejora en la comunicación entre la administración y los residentes del edificio "El Mirador".

2.3. Requerimientos funcionales

ID	Nombre Requerimiento	Descripción	Actor
RF-1	Gestión de residentes	Registrar, actualizar y eliminar información de residentes, asociándolos a departamentos específicos	Administrador
RF-2	Gestión de pagos de gastos comunes	Registrar pagos realizados por los residentes, Generar estados de cuenta individuales, Calcular automáticamente deudas y morosidad. Emitir comprobantes de pago	Administrador
RF-3	Gestión de Solicitudes y	Un módulo para que el residente ingrese quejas y	Sistema

	Reclamos	la administración responda	
RF-4	Sistema de Notificaciones	Automatización de correos electrónicos para avisos de cobro y alertas de morosidad	Residente / Administrador
RF-5	Control de morosidad	Identificar residentes con pagos pendientes. Generar alertas y notificaciones automáticas. Aplicar intereses o recargos por atraso (si corresponde)	Sistema
RF-6	Generación de informes	Generar reportes financieros (ingresos, deudas, morosidad).	Administrador

2.4. Requerimientos no funcionales

Usabilidad: El sistema debe ser intuitivo tanto para los residentes como para la administración del edificio.

Rendimiento y Disponibilidad: El sistema debe permitir un acceso transparente y a tiempo real de la información dentro del sistema (pagos, etc). Se sugiere que el tiempo de carga del sistema sea inferior a 2 segundos.

Fiabilidad: El sistema debe asegurar precisión en la cobranza y eficiencia de la gestión de recursos para garantizar la transparencia financiera.

Seguridad: El sistema debe garantizar la protección de los datos financieros tanto de los residentes como de la administración.

2.5. Supuestos y dependencias

Acceso a tecnología: La administración deberá contar con un computador y acceso a internet para poder actualizar los datos de pago y cobro en tiempo real.

Información de empresa: Se necesita la información de los servicios de luz, agua, etc. para poder ingresar las cifras al sistema y poder gestionar los pagos de manera automática

Capacitación: El sistema depende de que el personal esté dispuesto a capacitarse para usar el software de la manera indicada.

Comunicación: La administración es la encargada de gestionar los datos, la información se le hará llegar a los residentes directamente a su departamento o

por correo electrónico si así lo desean.

3. ESTILOS Y PATRONES ARQUITECTÓNICOS

3.1. Estilo arquitectónico adoptado (ej. monolítico, microservicios, SOA, capas)

- 1 Para el sistema se adoptó un Estilo Arquitectónico en Capas, el cual trabaja en conjunto con el patrón MVC (Modelo-Vista-Controlador). La estructura se divide en:
 - Capa de Presentación (Vista): Interfaz web y móvil que interactúa con los usuarios (administradores y residentes).
 - Capa de Lógica de Negocio (Controlador): Procesa las reglas del sistema, como el cálculo de gastos comunes, prorratesos y asignación de pagos.
 - Capa de Acceso a Datos (Modelo): Gestiona las consultas y la comunicación con la base de datos.
 - Capa de Persistencia: Almacenamiento seguro de la información en el motor Oracle Database.

3.2. Justificación del estilo según el contexto del sistema

Se eligió este estilo porque garantiza la mantenibilidad del código, asegura la escalabilidad del sistema y permite aislar la base de datos para cumplir con los estándares de seguridad y protección de datos exigidos.

3.3. Patrones de diseño aplicados (ej. patrón MVC, repositorio, etc.)

MVC: Lo usamos para separar las cosas. Las vistas (en PHP) solo muestran la interfaz, el controlador maneja la lógica y el modelo se enfoca en los datos. Así el Subsistema Financiero quedó súper estructurado.

DAO : Centraliza todas las consultas SQL hacia la base de datos Oracle. Nos sirve para que la lógica de negocio no toque directamente las tablas, haciendo el sistema más seguro y fácil de mantener si cambia la base de datos.

4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS

4.1. VISTA DE ESCENARIO

4.1.1. Propósito

El propósito de la vista de escenario es definir el alcance funcional del sistema desde la perspectiva de los usuarios finales. Sirve para identificar qué actores interactúan con la plataforma y qué acciones (casos de uso) pueden realizar, validando que se cubran todas las necesidades de administración y finanzas de la comunidad.

4.1.2. Actores

Administrador de la comunidad: Usuario principal encargado de la gestión global, configuración de torres, cálculo de gastos comunes, prorratesos y emisión de avisos de cobranza.

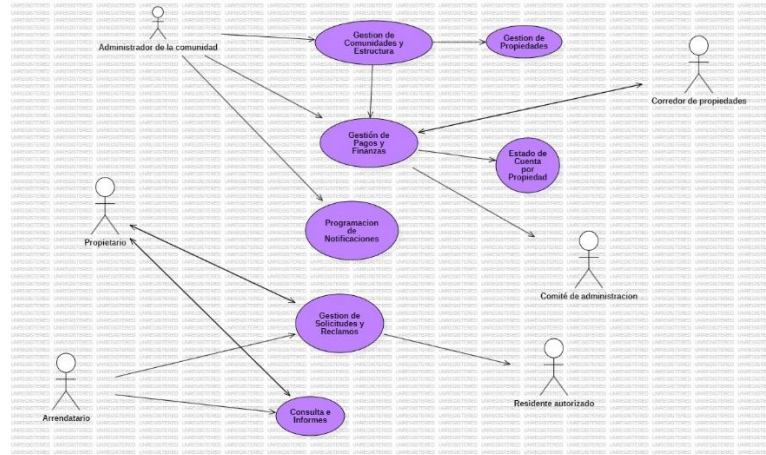
Propietario / Arrendatario: Residentes de la comunidad que acceden al sistema

para revisar sus estados de cuenta individuales, descargar colillas de cobro, reportar pagos y levantar solicitudes o reclamos.

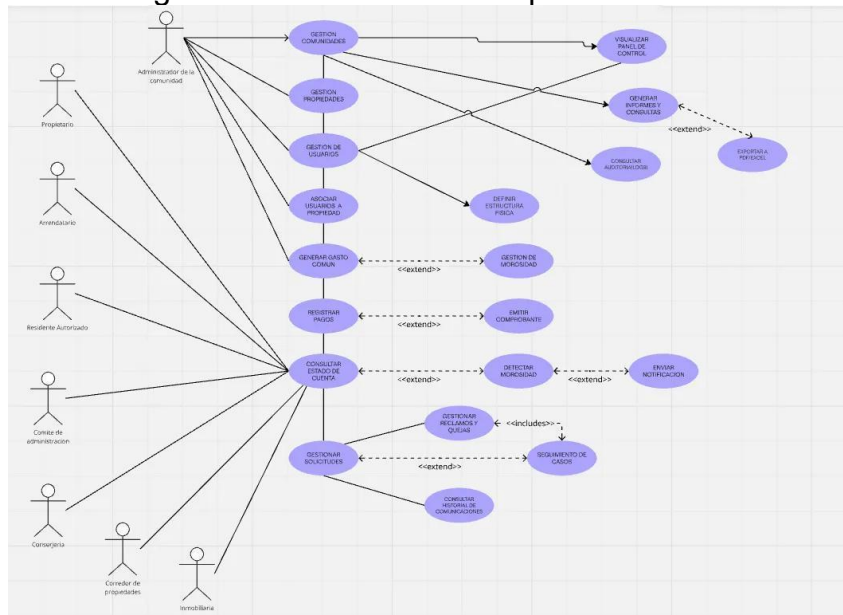
Comité de administración: Grupo supervisor que accede a informes financieros y auditorías de la gestión.

Sistemas Externos :Entidad externa (como Webpay/Transbank) que se conecta para procesar y validar las transacciones bancarias en línea de forma segura

4.1.3. Diagrama general de casos de uso



4.1.4. Diagrama de casos de uso específicos



4.1.5. Lista de casos de uso

Código	Nombre	Actores
CU-001	Registrar y Actualizar Residentes	Administrador
CU-002	Ingresar Cifras de Servicios Básicos del Edificio	Administrador
CU-003	Calcular Gasto Común y Prorratio	Sistema (Gestor Financiero)
CU-004	Enviar Notificación Automatizada de Cobro	Sistema
CU-005	Consultar Estado de Cuenta Individual	Residente / Propietario
CU-006	Procesar Pago de Gasto Común	Residente, Sistema Externo de Pagos
CU-007	Aplicar Intereses y Alertas de Morosidad	Sistema (Gestor Financiero)

CU-008	Ingresar Solicitud, Queja o Reclamo	Residente
CU-009	Atender y Responder Solicitudes de Copropietarios	Administrador
CU-010	Generar Informes y Reportes de Cobranza Financiera	Administrador

4.1.6. Especificación de casos de uso

Caso de Uso	Calcular Gasto Común y Prorrateo	Identificador: CU-003
Actores	Administrador, Sistema (Gestor Financiero).	
Tipo	Primario	
Referencias	RF-2 (Gestión de pagos de gastos comunes), RF-5 (Control de morosidad).	
Precondición	El Administrador ha ingresado al sistema las facturas de cobro de los servicios básicos correspondientes al mes en curso.	
Postcondición	El sistema genera y publica de manera exitosa los estados de cuenta individuales prorrateados para los departamentos de la comunidad.	
Descripción	Permite calcular de forma automática el monto exacto que debe pagar cada departamento por concepto de gastos comunes, distribuyendo proporcionalmente los costos globales del edificio.	
Resumen	Automatización del cálculo de la cuota mensual para eliminar errores humanos y asegurar la transparencia financiera.	

CURSO NORMAL

Nro.	Ejecutor	Paso o Actividad
1	Administrador	Ingresa al Panel de Control Financiero y selecciona la opción " Calcular Gasto Común ".
2	Sistema	Busca y consolida los montos totales de los servicios básicos e insumos registrados para el período actual.
3	Sistema	Invoca el método PRORRATEAR() de la clase GestorFinanciero para distribuir los costos de manera proporcional entre las propiedades.
4	Sistema	Revisa el historial en la base de datos Oracle para identificar departamentos con saldo pendiente y calcula deudas acumuladas.
5	Sistema	Genera el documento digital de Estado de Cuenta individualizado para cada departamento de la comunidad.
6	Sistema	Modifica el estado del período a "Emitido" y activa el envío de notificaciones automáticas por correo electrónico a los residentes (RF-4).

CURSO ALTERNATIVO

Nro.	Descripción de acciones alternas
2a	Gastos globales no registrados: Si el Administrador no ha ingresado los montos de los servicios del mes, el sistema bloquea el flujo, despliega un mensaje de alerta indicando <i>"Faltan registrar servicios básicos"</i> y regresa al panel principal.
4a	Residente registra morosidad previa: Si la propiedad arrastra deudas de meses anteriores , el método CALCULARMOROSIDAD() aplica automáticamente la tasa de interés o recargo legal por atraso configurada, sumándola al cobro total del mes.

Caso de Uso	Procesar Pago de Gasto Común	Identificador: CU-006
Actores	Residente / Cliente, Sistema, Sistema Externo de Pagos (Transbank/Webpay).	
Tipo	Primario	
Referencias	RF-2 (Registrar pagos realizados) , RF-4 (Avisos de cobro y alertas).	
Precondición	El sistema debe haber emitido previamente el Estado de Cuenta del mes y el residente debe autenticarse en la plataforma.	
Postcondición	El saldo de la propiedad se actualiza en tiempo real en la base de datos Oracle y se emite un comprobante electrónico de pago exitoso..	
Descripción	Modela el flujo electrónico mediante el cual un residente visualiza su deuda mensual y realiza la transacción financiera a través de una pasarela bancaria externa.	
Resumen	Recepción, validación y registro automatizado de la recaudación de fondos para mejorar el flujo de caja del edificio.	

CURSO NORMAL

Nro.	Ejecutor	Paso o Actividad
1	Residente	Inicia sesión en el portal web, accede a su perfil y consulta su Estado de Cuenta vigente.
2	Residente	Responde de forma afirmativa a la opción "¿Desea pagar ahora?" y selecciona su método de pago electrónico (Débito o Crédito).
3	Sistema	Redirige al residente de manera segura hacia la API del Sistema Externo de Pagos externo configurado.
4	Sistema Externo	Solicita las credenciales bancarias al usuario, procesa la transacción y ejecuta la validación bancaria de fondos.
5	Sistema Externo	Aprueba la transacción financiera y devuelve un código de confirmación de éxito al servidor web del edificio.
6	Sistema	Registra el ingreso del dinero en la base de datos Oracle, cambia el estado del departamento a "Al día" y envía el comprobante de pago electrónico al residente.

CURSO ALTERNATIVO

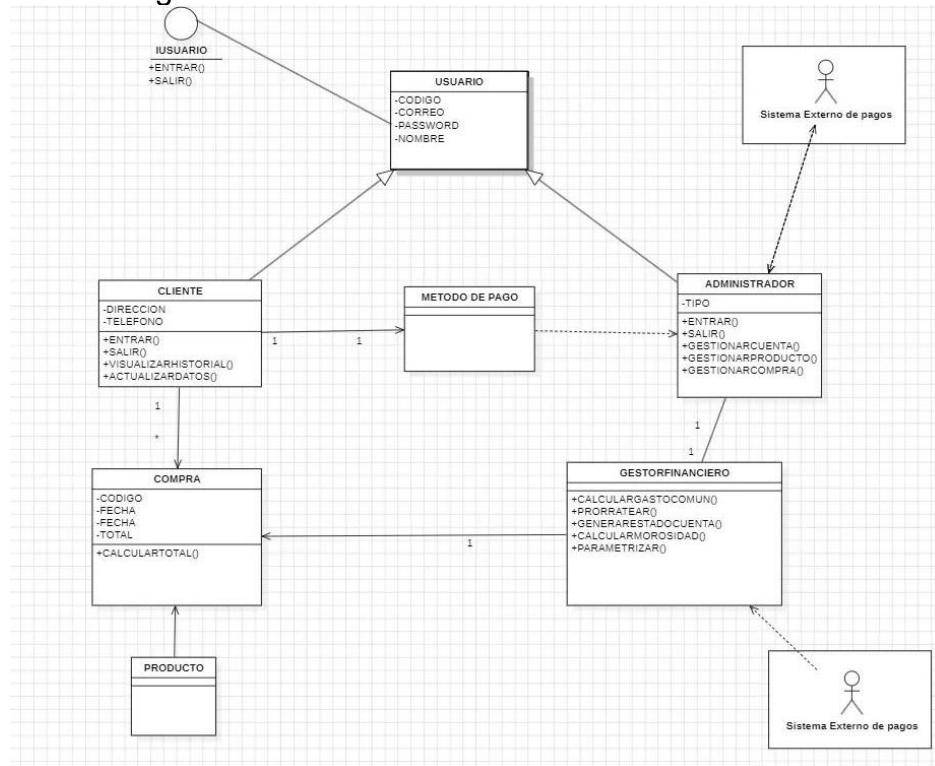
Nro.	Descripción de acciones alternas
2a	El residente rechaza pagar en el momento: Si el usuario selecciona "No" en la consulta de pago , el sistema finaliza la visualización y cierra el flujo de forma segura manteniendo la deuda vigente.
4a	Transacción bancaria rechazada: Si el usuario no cuenta con fondos suficientes o cancela la operación en la pasarela externa , el Sistema Externo devuelve un código de error. El software del edificio despliega en pantalla un mensaje informativo de "MOSTRAR ERROR" sin alterar los saldos en la base de datos.

4.2. VISTA LÓGICA

4.2.1. Propósito

Muestra la estructura interna y el diseño conceptual del sistema, detallando las clases, atributos, métodos y cómo se relacionan entre sí para cumplir con las reglas del negocio.

4.2.2. Diagrama de clases



4.2.3. Descripción diagrama de clases

El diseño se organiza a través de los siguientes componentes básicos:

- **Usuarios (USUARIO, CLIENTE, ADMINISTRADOR):** Se aplica herencia donde la clase madre USUARIO hereda sus datos a CLIENTE (residentes) y

ADMINISTRADOR. Ambos usan la interfaz IUSUARIO para entrar y salir del sistema.

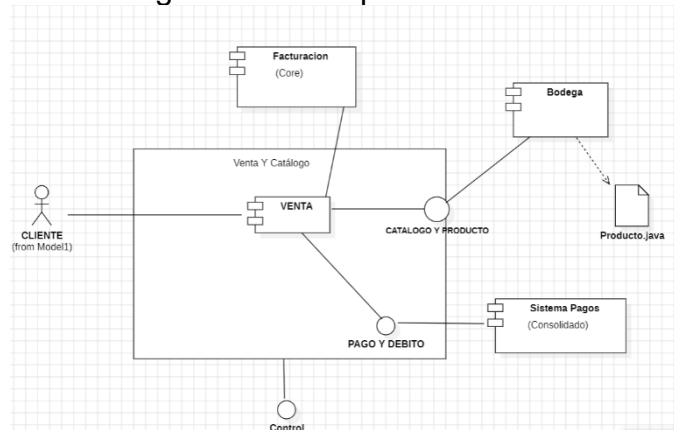
- Gestor Financiero (GESTORFINANCIERO): Es el motor del sistema. El administrador lo usa para calcular el gasto común, aplicar el prorateo, ver la morosidad y armar los estados de cuenta.
- Transacciones (COMPRA y METODO DE PAGO): Registra los movimientos financieros de cada cliente (como los cobros y pagos cargados) y calcula los totales.
- Sistema Externo de Pagos: Modela la conexión con la pasarela externa para procesar y validar los pagos del sistema.

4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO

4.3.1. Propósito

El propósito de este diagrama es mostrar la organización estructural y las dependencias físicas de los módulos del sistema. Permite visualizar cómo se dividen las responsabilidades del software mediante la implementación de los patrones MVC y DAO, detallando la separación entre la interfaz en PHP, las reglas de negocio financieras (como el prorateo) y el acceso seguro a los datos en Oracle, garantizando una arquitectura modular, desacoplada y escalable.

4.3.2. Diagrama de componente

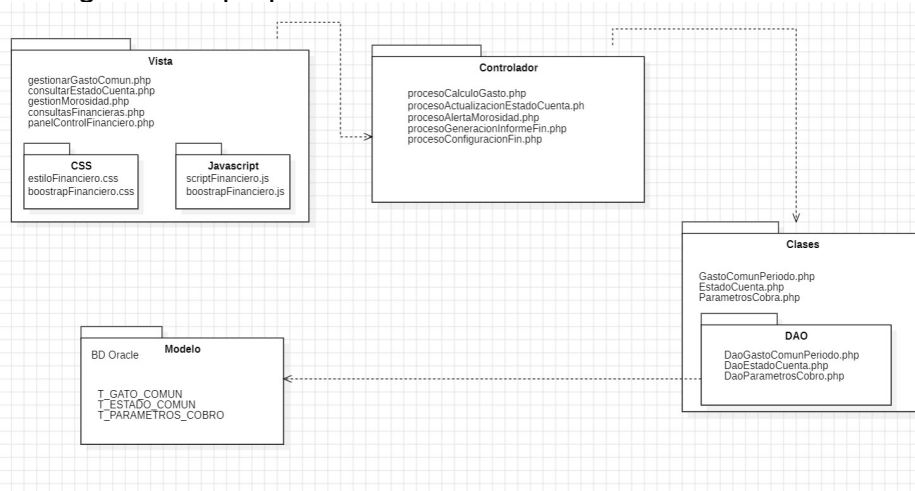


4.3.3. Descripción diagrama de componente

El diagrama de componentes describe la organización estructural del software del Edificio Mirador, evidenciando un bajo acoplamiento

mediante el uso de interfaces y la separación de responsabilidades

4.3.4. Diagrama de paquete



4.3.5. Descripción diagrama de paquete

El diagrama de paquetes organiza el sistema bajo el patrón arquitectónico MVC, permitiendo una separación clara entre la interfaz, la lógica de control y el acceso datos.

Paquete Vista: Contiene la interfaz de usuario. Agrupa archivos PHP para la gestión de gastos, morosidad y estados de cuenta, junto con subpaquetes de recursos estáticos (CSS y Javascript) que aseguran la experiencia visual y funcional del cliente y administrador.

Paquete Controlador: Actúa como intermediario. Contiene la lógica de procesamiento para eventos específicos, como el cálculo de gastos comunes (`procesoCalculoGasto.php`) y la generación de alertas de morosidad, canalizando las peticiones de la Vista hacia el Modelo.

Paquete Clases y DAO: Representa la lógica de negocio y el acceso a datos.

Clases: Define los objetos del dominio (Gasto Común, Estado de Cuenta).

DAO (Data Access Object): Subpaquete especializado que separa la lógica de negocio de las consultas técnicas a la base de datos, mejorando la mantenibilidad.

Paquete Modelo (BD Oracle): Representa la capa de persistencia final. Describe la estructura de tablas en la base de datos Oracle (como `T_GASTO_COMUN` y `T_ESTADO_COMUN`) donde se almacena la información física del sistema.

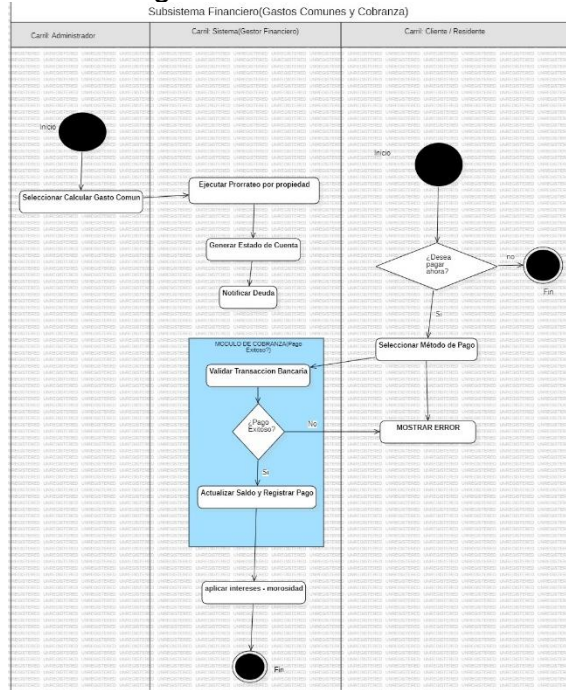
4.4. VISTA DE PROCESOS

4.4.1. Propósito

El propósito es representar el flujo de trabajo dinámico y el comportamiento del sistema ante un proceso específico.

En palabras simples, funciona como un diagrama de flujo avanzado que muestra los pasos secuenciales, las decisiones lógicas y las tareas en paralelo que ocurren desde que un proceso inicia hasta que termina.

4.4.2. Diagrama de actividad



4.4.3. Descripción diagrama de actividad

El diagrama modela el flujo del proceso Registro y Validación de Pagos mediante cuatro carriles de responsabilidad:

- Residente: Se autentica en la web, consulta su deuda, selecciona el monto y confirma el pago.
- Sistema (PHP): Orquesta el flujo, envía los datos a la pasarela externa y evalúa la respuesta bancaria.
- Sistema Externo (Webpay): Procesa de forma segura la transacción y la transferencia de fondos con el banco.
- Base de Datos (Oracle): Modifica las tablas T_GASTO_COMUN y T_PAGOS solo si aprueba el dinero.

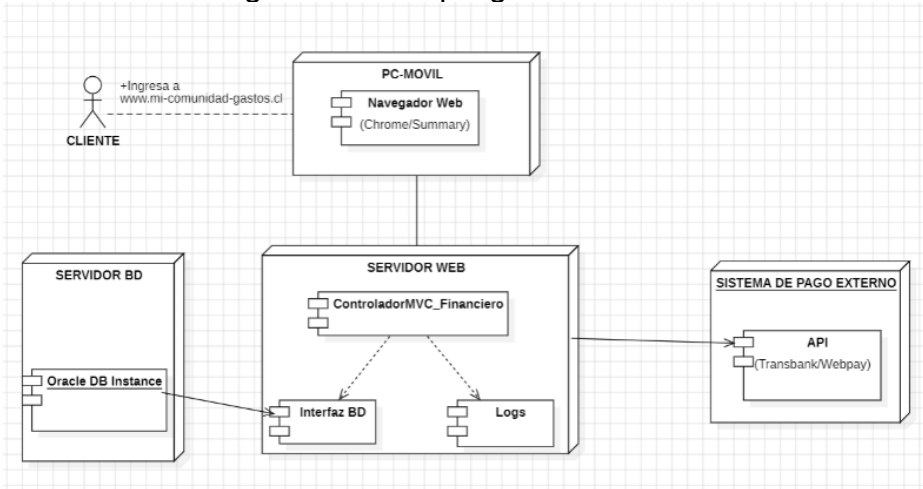
4.5. VISTA FÍSICA

4.5.1. Propósito

El propósito de este diagrama es representar la arquitectura física y la infraestructura tecnológica del sistema. Detalla la topología de la red mediante la distribución de los componentes de software en servidores independientes: un servidor web encargado de ejecutar la lógica en PHP (MVC) y un servidor aislado que aloja la base de datos Oracle (DAO). Además, describe el uso de protocolos seguros de comunicación como HTTPS y TCP/IP para la conexión con el usuario y la pasarela Webpay, garantizando un entorno escalable, eficiente y protegido contra

vulnerabilidades.

4.5.2. Diagrama de despliegue



4.5.3. Descripción diagrama de despliegue

El diagrama detalla la arquitectura física del sistema mediante cuatro nodos conectados por red: el Cliente Web (dispositivo del usuario), el Servidor de Aplicaciones (que procesa la lógica y el código PHP), el Servidor de Base de Datos (que almacena de forma aislada las tablas de Oracle) y el Servicio Externo (la pasarela Webpay). Toda la comunicación externa viaja encriptada mediante protocolo seguro HTTPS, mientras que la conexión con la base de datos se realiza de forma interna por TCP/IP, garantizando la máxima seguridad e integridad de la información financiera.

5. REQUISITOS DE CALIDAD

5.1. Propósito

El propósito de esta sección es detallar los atributos de calidad clave que gobernarán el comportamiento del software del Edificio Mirador. Con esto se busca asegurar que la plataforma no solo cumpla con sus tareas lógicas, sino que opere bajo estándares óptimos de velocidad, seguridad, precisión matemática y facilidad de uso, garantizando la transparencia administrativa y devolviendo la confianza a los copropietarios.

5.2. Atributos de calidad (por ejemplo: Usabilidad, Accesibilidad (WCAG), Rendimiento, Mantenibilidad, Seguridad Portabilidad)

ATRIBUTO DE CALIDAD	DESCRIPCION	JUSTIFICACIÓN
Usabilidad	Medida en que el software permite a los usuarios interactuar con la interfaz de forma intuitiva, minimizando la curva de aprendizaje y evitando confusiones en la navegación.	El sistema será utilizado por dos perfiles con realidades distintas: el Administrador y los Residentes o Propietarios, quienes poseen niveles de alfabetización digital muy diversos y requieren gestionar sus cuentas sin ayuda

		externa.
Rendimiento	Capacidad del sistema para responder a las peticiones de los usuarios, procesar flujos de datos y cargar elementos visuales dentro de un tiempo de espera optimizado.	Los procesos manuales actuales causan cuellos de botella administrativos. El nuevo software web debe entregar información financiera y validar pagos de forma ágil para asegurar una óptima experiencia en tiempo real.
Fiabilidad	Grado en que el sistema ejecuta sus funciones de manera correcta, exacta y consistente, manteniendo la integridad de la información procesada.	Cero error financiero: Garantiza que los algoritmos de prorrateo e intereses cobren el monto exacto, protegiendo el dinero de la comunidad. Consistencia con Webpay: Asegura que todo pago aprobado por el banco se registre de forma inmediata y sin fallos en la base de datos Oracle. Transparencia auditable: Evita la alteración o pérdida de estados de cuenta históricos, manteniendo la confianza de los copropietarios y el comité.
Seguridad	Capacidad del software para proteger los datos almacenados contra accesos maliciosos o no autorizados, asegurando el aislamiento de la información según el rol del usuario.	La plataforma centraliza datos financieros confidenciales de cobranza, registros de cuentas bancarias y datos personales/de contacto de todos los habitantes de los 160 departamentos .

5.3. Reglas y criterios de evaluación de calidad

Para comprobar de manera objetiva que el sistema cumple con los atributos definidos anteriormente, se aplicarán las siguientes reglas de negocio, métricas y herramientas de evaluación:

Métricas y Criterios de Aceptación:

- **Para Rendimiento:** El tiempo de carga de las pantallas principales (Dashboard de administración y Estado de cuenta del residente) debe ser estrictamente **inferior a 2 segundos** bajo condiciones normales de red. El procesamiento masivo del cálculo del gasto común mensual no debe superar los 5 segundos.

- **Para Usabilidad:** El diseño de interfaces debe registrar una **tasa de éxito superior al 90%** en las pruebas de tareas críticas (por ejemplo, que un residente complete el flujo de simulación de pago o que el administrador genere un informe de morosidad sin cometer errores de navegación).
 - **Para Fiabilidad:** Tolerancia cero al error matemático (**0% de margen de desviación**). Cualquier redondeo o cálculo de cuotas financieras debe cuadrar de manera exacta con los montos totales de las facturas de servicios ingresadas al sistema.
 - **Para Seguridad:** Exclusividad total de accesos basados en roles. Un usuario con perfil "Residente" tiene **0% de acceso permitido** a los métodos de modificación de datos de otros departamentos o configuraciones globales del dinero de la comunidad.
- Herramientas y Métodos que se utilizarán:**
- **Pruebas de Carga y Estrés:** Se utilizará la herramienta **Apache JMeter** para simular la concurrencia de múltiples residentes consultando y pagando sus gastos comunes al mismo tiempo, evaluando los tiempos de respuesta del servidor web y las consultas hacia la base de datos Oracle.
 - **Pruebas Heurísticas de Usabilidad:** Inspección detallada del prototipo utilizando una lista de verificación basada en los **10 principios de Jakob Nielsen**, complementada con sesiones de validación directa con usuarios reales del edificio para evaluar la claridad del lenguaje y el diseño.
 - **Pruebas Unitarias e Integrales Automatizadas:** Implementación de pruebas de software en el Controlador utilizando frameworks como **PHPUnit** (o JUnit según el entorno final de desarrollo) , diseñadas específicamente para validar que los algoritmos de prorrateo e intereses entreguen el resultado financiero numérico correcto de forma automatizada.
 - **Auditorías de Código e Inspecciones de Seguridad:** Revisiones periódicas de la gestión de sesiones del lado del servidor y auditorías sobre los Data Access Objects (DAO) para asegurar que las consultas SQL en Oracle estén protegidas contra vulnerabilidades de inyección y accesos cruzados.

6. PRINCIPIOS DE DISEÑO APLICADOS

6.1. Propósito

6.2. Principios de diseño (por ejemplo: abstracción, acoplamiento, cohesión, encapsulamiento, modularidad)

PRINCIPIO	DESCRIPCIÓN	APLICACIÓN EN EL SISTEMA
Cohesión	Cada módulo o clase tiene una única responsabilidad bien definida.	Los servicios están diseñados para realizar tareas específicas y no múltiples funciones

7. PROTOTIPO

7.1. Propósito

7.2. Mockups (imágenes con una breve descripción)

7.3. Justificar herramientas de prototipado

8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN

8.1. Propósito

8.2. Lista de verificación

8.3. Análisis y métricas de resultados

9. CONTROL DE VERSIONES

9.1. Propósito

9.2. Control de versión utilizado (justificar el tipo de control de versión utilizado (fecha, semántica o secuencial)

9.3. Justificar herramientas de versionamiento

7. CONCLUSIONES

8. BIBLIOGRAFÍA

9. ANEXOS

9.1. Carta Gantt